RUPEEFLOW FINANCE TRACKER

VISHVESHWARAPURAM COLLEGE OF SCIENCE
K.R road, V.V puram, Bangalore-560004

PROJECT REPORT ON

"RUPEEFLOW FINANCE TRACKER"

Submitted in partial fulfillment of the requirement of BCA VI semester course during the academic year 2025-26

HEAD OF THE DEPARTMENT
Mr. Shashidhar S B

Under the guidance of
Mr. Madhusudan Reddy C

Report Submitted by
[KOUSHIK R]
[U18ID23S0051]
Visveswarapura College, Department of Computer Science
K.R Road, Bengaluru – 560004

CERTIFICATE

Certified that the project work entitled "RupeeFlow Finance Tracker" is a bonafide work carried out by Koushik R [U18ID23S0051] in partial fulfilment for the award of the Degree of Bachelor

of Computer Applications of Bengaluru city University during the year 2025-26. The project report has been approved as it satisfies the academic requirements in respect of project work prescribed for the said degree.

(Mr. Madhusudan Reddy C)	(Mr. Shashidhar S B)
Signature of the Guide	Head of the Department

Date:

Signature of the Examiners

1. _____

2. _____

DECLARATION

I hereby declare that the work presented in the project entitled "RupeeFlow Finance Tracker" in partial fulfilment of the requirements for the award of the Degree of Bachelor of Computer Applications submitted in the Department of Computer Applications, Visveswarapura College, is an authentic record of my own work carried out under the supervision and guidance of **Mr. Madhusudhan C**

The matter embodied in this project work has not been submitted for the award of any other degree or diploma.

Date: Koushik R
Place: Bangalore U18ID23S0051

ACKNOWLEDGEMENT

The satisfaction and sense of accomplishment that accompany the successful completion of any task would be incomplete without acknowledging the people who made it possible, and whose constant guidance and encouragement crowned my effort with success.

I express my sincere gratitude to my respected Head of the Department, Mr. Shashidhar S.B, for providing me with the facilities and a supportive environment to carry out this project work.

I am deeply thankful to my guide, Mr. Madhusudan Reddy C, and our respected HOD, Mr. Shashidhar S B, for their consistent supervision, valuable suggestions, and constant motivation throughout the development of this project. Their guidance has been instrumental in shaping this work.

I extend my heartfelt gratitude to my parents, who have always encouraged me with their blessings and support. Finally, I would like to thank all my friends and the teaching and non-teaching staff of the Department of Computer Applications for their timely help, ideas, and encouragement that helped me throughout the completion of this project.

Koushik R

ABSTRACT

RupeeFlow Finance Tracker is an INR-first web application developed to help individuals record, organize, and analyze personal finances in one practical platform. It replaces scattered methods such as paper notes, spreadsheets, UPI histories, and raw bank statements with an interactive system that gives users a clear view of their financial health.

The system supports local registration and login, income and expense management, monthly budgets, savings goals, recurring transactions, notifications, reports, data export, and backup. Its dashboard shows balance, monthly income and spending, savings rate, top spending category, bills due, and a 30-day cash-flow forecast. Users can import selectable-text PDF bank statements or CSV files; transactions are extracted, categorized using India-relevant rules, and checked for duplicates before storage.

The application is primarily a React single-page system built with HTML, CSS, JavaScript, Vite, React Router, Recharts, PDF.js, and IndexedDB. Financial collections remain local to the browser through an asynchronous storage adapter. A small Node.js HTTP service uses Nodemailer and authenticated SMTP only for password-reset email OTP delivery; SMTP credentials remain outside frontend code in a private environment file. All monetary values are displayed only in Indian Rupees (INR).

INDEX

1. INTRODUCTION

1.1 Overview

Managing personal finances is a task that affects nearly everyone, yet it is one that many people struggle to do consistently. Income arrives from different sources, expenses occur across many categories, and financial commitments such as rent, subscriptions, and loan repayments recur every month. In the absence of a single organized system, this information becomes scattered across bank statements, mobile wallet histories, paper receipts, and memory. As a result, individuals often lose track of where their money goes, how much they are saving, and whether they are staying within their means.

The RupeeFlow Finance Tracker is a web-based application designed to address this problem. It provides a centralized digital platform where a user can record every income and expense, categorize transactions, set monthly spending limits, define savings goals, and review clear visual summaries of their financial activity. By bringing all of this together in one place, the application helps users understand their financial behavior and make better-informed decisions.

The application is built as a React single-page application supported by a small OTP email service. React Context providers manage authentication state, finance data, and settings; pure JavaScript modules handle calculations, recurring rules, statement parsing, duplicate detection, export, and formatting. Financial records are persisted locally in IndexedDB, while a Node.js HTTP endpoint uses Nodemailer and SMTP for password-reset verification codes.

1.2 Problem Statement

Traditional approaches to personal money management suffer from several recurring problems:

- Financial information is fragmented across multiple sources, making it difficult to see the complete picture.
- Manual tracking using notebooks or spreadsheets is time-consuming and error-prone.
- Without categorization, users cannot identify which areas of spending consume the most money.
- There is usually no mechanism to set budgets and receive a warning when spending approaches or exceeds a limit.
- Savings goals are rarely tracked in a structured way, so progress toward them is hard to measure.
- Recurring payments must be remembered and entered manually every time, which is easy to forget.

The RupeeFlow Finance Tracker provides an integrated solution to these problems through automation, categorization, visualization, and structured goal tracking.

1.3 Objectives

The key objectives of the project are as follows:

1. To provide a single, centralized platform for recording and organizing personal income and expenses.
2. To allow users to categorize transactions and analyze spending patterns by category.
3. To enable the creation of monthly budgets and provide visual indicators and alerts for budget usage.
4. To support the definition and tracking of savings goals with progress indicators.
5. To automate recurring transactions such as salary, rent, and subscriptions.
6. To present a clear dashboard and detailed reports using charts and summaries.
7. To allow users to export, back up, restore, and import their financial data from PDF bank statements or CSV files.
8. To ensure the application is secure, private, responsive, and easy to use.

1.4 Scope of the Project

The project focuses on personal finance management for Indian users. Its scope includes local account access, INR-only transaction management, PDF and CSV bank-statement import, duplicate prevention, budgeting, savings goals, recurring transactions, India-relevant categories, notifications, analytics, data export and backup, and light, dark, or system themes. The application is designed for students, working professionals, freelancers, and households seeking clearer control over UPI spending, EMIs, bills, and savings.

The current version stores each user's financial collections locally in browser IndexedDB and supports JSON backup and restore. It extracts transactions from supported selectable-text PDF statements and provides email OTP verification for password reset. OTP codes expire after ten minutes, permit five attempts, enforce a sixty-second resend delay, and produce one-time reset tokens. Cloud synchronization, production-grade account authentication, and live bank feeds remain future enhancements.

1.5 Existing System

In the absence of a dedicated tool, most people manage their finances using one or more of the following methods: keeping mental notes, writing in a physical notebook, maintaining a spreadsheet, or relying on the transaction histories provided by individual banks and payment apps. Each of these methods has significant drawbacks. Mental notes are unreliable and easily forgotten.

Notebooks cannot perform calculations or produce summaries. Spreadsheets require manual setup and formula maintenance and offer no guidance or alerts. Bank histories are fragmented across institutions and present raw lists without categorization, budgeting, or goal tracking.

The common weakness across all of these existing approaches is that they are passive records rather than active tools. They store information but do not help the user interpret it, plan ahead, or stay disciplined.

1.6 Proposed System

The proposed RupeeFlow Finance Tracker provides an active, integrated system for Indian personal finance. It records and categorizes income and expenses, imports PDF bank statements, rejects duplicate entries, compares spending against budgets, tracks savings goals, automates recurring entries, and presents forecasts and visual reports. INR-only displays and categories for UPI, EMI, transport, food, utilities, and other common needs make the system practical for its intended users.

1.7 Feasibility Study

A feasibility study was carried out to confirm that the project is practical to build and use.

- Technical feasibility - The application uses widely supported, free web technologies: React, Vite, React Router, Recharts, PDF.js, and browser IndexedDB. It runs in a modern browser without a dedicated application server or special hardware.
- Economic feasibility - All frameworks and libraries used are open source, while IndexedDB provides built-in local database storage without a paid database service.
- Operational feasibility — The interface is simple and intuitive, requiring no training. Users already familiar with using a web browser can operate the application immediately, confirming operational feasibility.

2. PROJECT OVERVIEW

2.1 Description

RupeeFlow Finance Tracker is a single-page application for managing finances in Indian Rupees. After signing in, the user sees a dashboard summarizing balance, current-month income and expense, savings rate, top spending, upcoming bills, and a 30-day forecast. Navigation provides dedicated screens for income, expenses, transactions and statement import, budgets, goals, recurring rules, reports, and profile settings.

Every screen is built around a consistent set of data collections — transactions, budgets, goals, and recurring rules — which are stored on the user's device and read through a shared data layer. Changes made on one screen are immediately reflected wherever that data appears, so the dashboard, reports, and individual pages always remain in sync.

2.2 Target Users

The application is intended for a broad range of users who want better visibility and control over their money:

- Students, who need to manage limited budgets and track everyday spending.
- Working professionals, who want to monitor salary, expenses, and savings.
- Freelancers, who deal with irregular income from multiple sources.
- Small business owners, for basic personal finance use.
- Anyone who wishes to develop disciplined financial habits.

2.3 Modules of the System

The system is organized into the following major modules:

Module	Purpose
Local User Access	Local registration, login/logout, email OTP password reset, and profile management.
Dashboard	Balance, savings rate, top spending, bills due, budgets, trends, and 30-day forecast.
Income Management	Recording, editing, and categorizing income entries.
Expense Management	Recording, editing, and categorizing expenses.
Transactions & Import	Search, filter, sort, bulk actions, export, PDF/CSV import, categorization, and duplicate prevention.
Budget Management	Setting monthly category limits and tracking spending against them.
Savings Goals	Defining targets and tracking progress toward them.
Recurring Transactions	Automating periodic income and expenses.

Module	Purpose
Reports & Analytics	Visualizing financial data through charts and summaries.
Settings	Theme control, INR-only display, export, IndexedDB-backed backup, restore, and reset.
Custom Category Management	Creates and persists per-user income and expense categories, prevents blank or duplicate names, and shares them across transaction, recurring, and budget forms.

2.4 Technologies Used

The project is a browser-based React application supported by modular state, business-logic, statement-processing, visualization, and IndexedDB persistence layers.

Component	Technology
Frontend structure	HTML5
Frontend styling	CSS3 (responsive design tokens; light, dark, and system themes)
Programming language	JavaScript (ES6+)
Frontend library	React 19
Frontend build tool	Vite 8
Routing	React Router
Charts	Recharts
PDF text extraction	PDF.js (pdfjs-dist)
Statement processing	Custom parser for supported PDF and CSV formats
Primary database	IndexedDB (rupeeflow database)
OTP email service	Node.js HTTP server, Nodemailer, and authenticated SMTP
State management	React Context
Quality checks	ESLint and Vite production build

2.5 Software Development Methodology

The project was developed following an iterative and incremental model derived from the Software Development Life Cycle (SDLC). Rather than building the entire application in a single pass, the system was developed in stages, with each stage producing a working increment that was then reviewed and refined. This approach allowed problems to be identified early and features to be improved progressively.

The phases followed during development were:

9. Requirement analysis — identifying the problems with existing methods and defining the features the system must provide.
10. Design — planning the architecture, the data model, the module structure, and the user interface.
11. Implementation — building the features incrementally, starting with the core (transactions and dashboard) and progressively adding budgets, goals, recurring transactions, reports, and settings.
12. Testing — verifying each feature, validating input handling, and checking responsiveness.
13. Refinement — polishing the user interface, adding the theme system, and improving usability based on review.

2.6 Hardware and Software Requirements

2.6.1 Software Requirements

Software	Requirement
Operating System	Windows, macOS, or Linux
Web Browser	Any modern browser (Chrome, Firefox, Edge, Safari)
Runtime (for development)	Node.js
Code Editor (for development)	Visual Studio Code or any text editor

2.6.2 Hardware Requirements

Component	Minimum Requirement
Processor	Dual-core processor or higher
RAM	2 GB or more
Storage	100 MB free space for development files
Display	Any standard display; responsive down to mobile sizes

3. SYSTEM ARCHITECTURE

3.1 Architectural Overview

RupeeFlow Finance Tracker follows a layered, component-based architecture. Four main layers separate presentation, application/state, business logic, and storage. PDF extraction and statement parsing operate within the business-logic layer, while IndexedDB provides persistent structured storage through a dedicated adapter.

Layer	Responsibility	Key Files
Presentation	Renders screens and handles user interaction.	pages/, components/
Application / State	Holds shared state and exposes operations to the UI.	store/ (React Contexts)
Business Logic	Pure functions that derive totals, trends, and alerts.	lib/selectors.js, lib/recurring.js
Storage	Persists per-user collections through an IndexedDB adapter with migration and fallback.	lib/storage.js

3.2 Layered Design

3.2.1 Presentation Layer

The presentation layer consists of React components organized into pages and reusable UI elements. Each page corresponds to a major feature — for example, the Dashboard page, the Transactions page, or the Budgets page. Reusable components such as the application shell (navigation and layout), the transaction list, modals, progress bars, and charts are shared across pages to maintain a consistent look and avoid duplication.

3.2.2 Application / State Layer

The application layer uses React Context providers. Authentication Context manages local users and sessions and coordinates the email OTP recovery API. Data Context loads transactions, budgets, goals, and recurring rules; exposes create, update, delete, bulk-import, backup, restore, and reset operations; and prevents duplicate transactions. Settings Context manages theme preferences and enforces INR formatting throughout the application.

3.2.3 Business-Logic Layer

The business-logic layer is a collection of pure JavaScript functions that contain no user-interface code. These functions take the raw data collections as input and compute derived values such as monthly totals, category-wise spending, budget status, savings progress, monthly trends, and the list of active notifications. Because these functions are pure, they always produce the same output for the same input, which makes the logic predictable and easy to test independently of the interface.

3.2.4 Storage Layer

All financial reads and writes pass through the storage abstraction in storage.js. The current IndexedDbStorageAdapter stores per-user collections in the rupeeflow browser database and automatically migrates compatible records from the earlier local-storage format. If IndexedDB is unavailable, the application uses a local-storage fallback. Feature modules depend on the adapter interface rather than storage details.

PDF.js extracts positioned text from uploaded statements. The statement parser rebuilds transaction rows, recognizes supported date and debit/credit formats, infers transaction type where required from running-balance changes, applies India-focused categorization rules, and submits valid records to Data Context for duplicate-safe bulk insertion.

3.3 Data Flow

A typical interaction flows through the layers as follows:

14. The user performs an action in the presentation layer, such as adding an expense.
15. The page calls a function exposed by the Data Context in the application layer.
16. The Data Context updates its state and writes the change through the storage layer.
17. The storage adapter writes the updated per-user collection to IndexedDB and returns control to the Data Context.
18. Business-logic functions recompute derived values such as totals and budget status.
19. The presentation layer re-renders automatically to reflect the updated data.

3.4 Architecture Diagram (Description)

The architecture combines a browser application with a narrowly scoped OTP email service. Pages and reusable components form the presentation layer; Authentication, Data, and Settings contexts form the state layer; selectors, recurring logic, duplicate fingerprints, PDF extraction, statement parsing, and export utilities form the business layer; IndexedDB forms the finance persistence layer. Password recovery calls Node.js endpoints that generate, verify, expire, and consume OTP credentials before the local password record is changed.

4. FUNCTIONAL REQUIREMENTS

This chapter describes what the system must do. Functional requirements specify the behaviour of each module and the operations available to the user.

4.1 User Authentication

- The system shall allow a new user to register with a name, username, email, and password.
- The system shall validate registration details and prevent duplicate local email or username records.
- The system shall ensure that both the email address and the username are unique, rejecting duplicates.
- The system shall allow a registered user to log in using either their email or username, and to log out securely.
- The system shall reset a forgotten password only after a six-digit email OTP is verified. Codes shall expire after ten minutes, allow five attempts, enforce a resend delay, and issue a one-time reset token.
- The demonstration shall clearly treat browser-based account storage as local-only and not as production-grade authentication.
- The system shall keep each local user’s financial collections separated by user identifier in browser storage.

4.2 Transaction Management

- The system shall allow the user to add income and expense transactions with an amount, category, date, and optional note.
- The system shall allow the user to edit and delete existing transactions.
- The system shall allow the user to categorize each transaction and create new income or expense categories from transaction, recurring-rule, and budget forms. Custom category names shall reject blank and duplicate values and remain saved per user.

4.3 Transaction History and Statement Import

- The system shall display all transactions in a list.
- The system shall allow searching by note or category.
- The system shall allow filtering by type, category, date range, and amount.
- The system shall allow sorting by date, amount, or category.
- The system shall allow selecting multiple transactions and deleting them together.

- The system shall allow exporting transactions to CSV and Excel formats.
- The system shall import transactions from unlocked selectable-text PDF bank statements and CSV files, automatically categorize supported rows, and report imported or skipped counts.

4.4 Duplicate Prevention and Budget Management

- The system shall reject duplicate manual entries, edited entries that duplicate another record, repeated recurring entries, and duplicates within or across statement imports.
- The system shall track actual spending against each budget.
- The system shall display the percentage of budget used with color-coded indicators.
- The system shall warn the user when a budget is nearly reached or exceeded.

4.5 Savings Goals

- The system shall allow the user to create savings goals with a target amount and optional target date.
- The system shall allow the user to add or withdraw funds from a goal.
- The system shall display progress toward each goal using a percentage and a progress indicator.
- The system shall indicate when a goal has been reached.

4.6 Recurring Transactions

- The system shall allow the user to define recurring income or expenses with a frequency (weekly, monthly, or yearly).
- The system shall automatically generate transactions from recurring rules when they are due.
- The system shall allow the user to pause, resume, edit, or delete a recurring rule.

4.7 Reports & Analytics

- The system shall display income-versus-expense comparisons over a selectable time range.
- The system shall display a net savings trend over time.
- The system shall display category-wise spending breakdowns.
- The system shall allow exporting reports to CSV and Excel and printing a styled report to PDF through a dedicated print document that waits for styles, fonts, and charts before opening the print dialog.

4.8 Notifications

- The system shall notify the user about exceeded or nearly-exceeded budgets.
- The system shall remind the user about upcoming recurring bills.
- The system shall notify the user when a savings goal milestone is reached.

4.9 Settings & Personalization

- The system shall allow the user to switch between light, dark, and system themes.
- The system shall display all monetary values exclusively in Indian Rupees (INR).
- The system shall allow the user to export all data and create and restore backups.

4.10 Non-Functional Requirements

Requirement	Description
Usability	The interface must be clean, intuitive, and require no training.
Performance	Screens and computations must respond instantly for typical data volumes.
Responsiveness	The layout must adapt to mobile, tablet, and desktop screens.
Privacy	Financial collections must remain local to the user's browser unless a future cloud service is explicitly introduced.
Maintainability	Code must be modular and clearly separated by responsibility.
Portability	The application must run on any modern web browser.

4.11 Use Case Descriptions

The following use cases describe the principal interactions between the user (the actor) and the system.

Use Case 1: Add a Transaction

Field	Detail
Actor	Registered user
Precondition	The user is logged in.
Main flow	The user opens the income or expense screen, enters the amount, category, date, and optional note, and confirms. The system validates the input and saves the transaction.
Postcondition	The transaction appears in the list and totals are updated.

Use Case 2: Set a Budget

Field	Detail
Actor	Registered user

Field	Detail
Precondition	The user is logged in.
Main flow	The user selects a month, chooses an expense category, and enters a monthly limit. The system stores the budget and begins tracking spending against it.
Postcondition	The budget appears with a progress indicator that updates as expenses are added.

Use Case 3: Create and Fund a Savings Goal

Field	Detail
Actor	Registered user
Precondition	The user is logged in.
Main flow	The user creates a goal with a name, target amount, and optional date, then adds funds to it over time. The system updates the progress on each contribution.
Postcondition	The goal’s progress ring reflects the amount saved; the goal is marked reached when the target is met.

5. DETAILED FEATURE DESCRIPTION

This chapter describes each current feature of RupeeFlow Finance Tracker, including India-focused transaction handling, statement import, local database storage, duplicate prevention, and responsive interaction design.

5.1 Local User Access and Email OTP Recovery

The access module lets a user register with a name, email address, and password, then sign in using the stored account on the same browser profile. Duplicate email addresses are rejected. Password recovery follows three steps: request a code at the registered email, verify the six-digit OTP, and choose a new password. The interface reports expiry, incorrect attempts, resend timing, and email-delivery guidance.

Accounts and sessions remain local to the browser, while OTP generation and email delivery run in a small Node.js service so SMTP credentials are never exposed in frontend code. Nodemailer sends through authenticated Gmail SMTP. Codes are stored only as salted hashes in server memory, expire after ten minutes, allow five attempts, and are replaced by single-use reset tokens after successful verification. This improves recovery integrity but does not convert local authentication into production-grade identity management.

5.2 Dashboard

The dashboard provides balance, current-month income and expense, savings rate, top spending category, bills due, and a 30-day cash-flow forecast. It also presents income-versus-expense trends, category spending, budget status, recent transactions, and notifications. These summaries update immediately when records are added, edited, deleted, generated, restored, or imported.

5.3 Income Management

The income module records money from sources relevant to Indian users, such as salary, freelance work, interest, investments, gifts, refunds, and other income. Each entry contains amount, category, date, and an optional note. All amounts are formatted in INR, duplicate entries are blocked, and users can add reusable custom income categories when the predefined list is insufficient.

5.4 Expense Management

The expense module records spending in categories such as food and dining, groceries, transport, fuel, rent and housing, utilities, EMI and loans, healthcare, shopping, education, entertainment, UPI and wallet activity, and other expenses. Monthly and all-time totals update immediately, duplicate entries are blocked, and users can add reusable custom expense categories for practical personal needs.

5.5 Transaction History

The transaction screen combines income and expenses with search, filtering, sorting, bulk selection, deletion, and export. Its statement importer accepts CSV or unlocked selectable-text PDF files. PDF.js extracts statement text; supported Indian bank formats are parsed into dated debit or credit records, assigned categories from narration keywords, and previewed through an import result. Duplicate fingerprints based on date, type, amount, category, and normalized note prevent repeated records within the file or against existing data.

5.6 Budget Management

The budgeting module allows the user to set a monthly spending limit for each expense category. As transactions are recorded, the system continuously compares actual spending against each limit and displays the result using color-coded progress bars: green when spending is comfortably within the limit, amber when it approaches the limit, and red when the limit is exceeded. Summary figures show the total amount budgeted, the total spent, and the remaining amount. This proactive feedback helps the user prevent overspending before it happens.

5.7 Savings Goals

The savings-goals module helps the user save with purpose. The user creates a goal — such as an emergency fund, a new laptop, a vacation, or a vehicle — and specifies a target amount and an optional target date. Each goal is displayed with a circular progress ring showing the percentage saved, the amount still required, and, where a date is set, the number of days remaining. The user can add funds to or withdraw funds from a goal, and the system clearly marks a goal as reached once its target is met.

5.8 Recurring Transactions

Many financial events repeat on a regular schedule. The recurring-transactions module lets the user define these once — for example, a monthly salary, monthly rent, a weekly expense, or a yearly subscription — by specifying the amount, category, frequency, and start date. The system then automatically generates the corresponding transactions when they fall due, sparing the user from repetitive manual entry. Recurring rules can be paused, resumed, edited, or deleted at any time.

5.9 Reports & Analytics

The reports module turns raw data into understanding. It presents the user’s finances over a selectable range of three, six, or twelve months. A bar chart compares income and expenses month by month, a line chart shows the net savings trend, and a pie chart breaks spending down by category. Summary figures report total income, total expenses, net result, and average monthly spend for the selected range. Reports can be exported to CSV and Excel or printed to PDF. Printing

uses an isolated, fully styled document and waits for content to load, preventing blank print previews and making reports suitable for record-keeping and sharing.

5.10 Notifications

The notification system keeps the user informed without requiring them to search for problems. A bell icon in the top bar displays the number of active alerts. These include warnings when a budget is nearly or fully consumed, reminders about recurring bills that are due soon, and congratulatory messages when a savings milestone or goal is reached. Each alert can be dismissed, and dismissed alerts do not reappear.

5.11 Settings & Personalization

Profile settings allow light, dark, or system-driven themes while INR remains the only supported currency. Users can export transactions, create a complete JSON backup, restore that backup, or reset local finance data. Data-entry dialogs are rendered directly under the document body through a React portal. They use transparent overlays, viewport-aware height, fixed headers, internal scrolling, responsive widths, and wrapping actions so fields and submit buttons remain reachable without clipping on smaller screens.

6. DATA MODEL & STORAGE

This chapter describes how RupeeFlow Finance Tracker represents and stores transactions, budgets, savings goals, and recurring rules together with local account and preference data.

6.1 Data Entities

6.1.1 Transaction

A transaction represents a single movement of money, either income or an expense. It is the central entity of the application.

Field	Type	Description
id	String	Unique identifier for the transaction.
type	String	“income” or “expense”.
amount	Number	The monetary value of the transaction.
category	String	The category the transaction belongs to.
date	String	The date of the transaction (ISO format).
note	String	An optional description.
recurringId	String	Set if generated from a recurring rule.

6.1.2 Budget

A budget defines a spending limit for one expense category in a particular month.

Field	Type	Description
id	String	Unique identifier for the budget.
month	String	The month the budget applies to (e.g. 2026-06).
category	String	The expense category being limited.
limit	Number	The maximum amount allowed for the month.

6.1.3 Savings Goal

A savings goal records a target the user is saving toward.

Field	Type	Description
id	String	Unique identifier for the goal.
name	String	The name of the goal.
target	Number	The target amount to be saved.
saved	Number	The amount saved so far.
targetDate	String	An optional target completion date.

6.1.4 Recurring Rule

A recurring rule describes a transaction that repeats on a schedule.

Field	Type	Description
id	String	Unique identifier for the rule.
type	String	“income” or “expense”.
amount	Number	The amount of each generated transaction.
category	String	The category of the transaction.
frequency	String	“weekly”, “monthly”, or “yearly”.
startDate	String	The date from which the rule begins.
paused	Boolean	Whether the rule is currently paused.

6.2 Relationships

A recurring rule generates transactions linked through recurringId. A budget belongs to a category and month, and its status is computed from matching expenses. Goals contribute to savings summaries. Each local user’s finance collections are stored under separate IndexedDB keys. Transaction fingerprints enforce uniqueness across manual entry, editing, recurring generation, and statement import.

6.3 Storage Mechanism

Financial collections are persisted in the browser’s IndexedDB database named rupeeflow, in a collections object store keyed by user and collection name. The adapter migrates older finance records from localStorage when needed. A localStorage adapter remains as a compatibility fallback for browsers where IndexedDB cannot be opened.

Pages do not access IndexedDB directly. They call asynchronous operations exposed by Data Context, which uses the storage adapter to list and write collections. This isolates persistence details, keeps all screens synchronized, and permits a future cloud or server adapter without rewriting feature pages.

```
createStorage() -> IndexedDbStorageAdapter (preferred) -> rupeeflow IndexedDB / collections
store -> LocalStorageAdapter (fallback). Core methods: list(), write(), getMeta(), setMeta().
Per-user metadata includes dismissed alerts and custom category lists.
```

6.4 Database Design and Extensibility

IndexedDB gives the current project a real structured browser database with asynchronous operations, larger capacity than localStorage, and per-user collection keys. For multi-device

production use, the same adapter boundary can later target a managed database such as PostgreSQL, MongoDB, Firebase, or Supabase together with secure server-side authentication.

6.5 Category Reference

Transactions begin with separate predefined income and expense category lists. Users may add custom categories from relevant forms; these are stored per user as customCategories metadata and become available in transactions, recurring rules, budgets, filters, and reports. Blank and case-insensitive duplicate names are rejected.

Income Categories	Expense Categories
Salary	Food
Freelance	Transport
Investments	Rent / Housing
Gifts	Utilities / Bills
Other	Entertainment
	Healthcare
	Shopping
	Other

6.6 Sample Data

The following table illustrates how a few transactions are represented in the system. This sample shows the variety of types, categories, and notes that the application stores.

Date	Type	Category	Amount	Note
2026-06-01	Income	Salary	45,000	Monthly salary
2026-06-03	Expense	Rent	12,000	June rent
2026-06-05	Expense	Food	850	Groceries
2026-06-09	Expense	Transport	300	Fuel
2026-06-12	Income	Freelance	6,000	Design project
2026-06-15	Expense	Entertainment	500	Movie

From data such as this, the application derives every summary it presents: the balance, the monthly income and expense totals, the category breakdown, the budget status, and the trends shown in the reports.

7. JAVASCRIPT / MODULE DESIGN

This chapter describes how the application's code is organized into modules and how those modules cooperate. The project is structured so that each module has a clear and single responsibility.

7.1 Project Structure

The source code is organized into folders by responsibility:

```
src/   store/  State providers (Auth, Data, Settings)   hooks/  INR formatting logic   lib/
IndexedDB storage, domain helpers, selectors, recurring, PDF extraction, statement parsing,
export   pages/  One file per screen   components/  Shell, icons, shared UI and transaction
list   styles/  Responsive design tokens and themes
```

7.2 State Modules (store)

- Authentication Context - manages registration, login, logout, the current session, email OTP requests and verification, and verified password reset.
- Data Context — loads the user's collections and exposes operations to add, update, and delete records, materializing recurring transactions when due.
- Settings Context - manages light, dark, and system themes and enforces INR-only money display.

7.3 Library Modules (lib)

- storage.js - implements the IndexedDB storage adapter, automatic legacy migration, and localStorage fallback.
- domain.js - defines India-relevant categories, colors, INR formatting, date helpers, and transaction fingerprints.
- selectors.js - computes totals, category spending, budgets, goals, trends, alerts, savings rate, and cash-flow forecast.
- recurring.js — converts recurring rules into transactions when they become due.
- pdfStatement.js and statementImport.js - extract selectable PDF text, parse supported statement rows, infer debit or credit, categorize narrations, and prepare duplicate-safe imports; export.js handles CSV, Excel, and print-to-PDF output.

7.4 Hooks

The useMoney hook returns a shared INR formatter. Components call it to display Indian Rupee values consistently without passing formatting rules through every component.

7.5 Design Principles

20. Separation of concerns — presentation, state, logic, and storage are kept apart.

21. Single source of truth — shared state lives in contexts, not scattered across components.
22. Pure functions — calculations are implemented as predictable, testable pure functions.
23. Program to an interface - features depend on the storage adapter, allowing IndexedDB to be replaced without changing feature pages.
24. Reusability — common UI and logic are factored into shared components and hooks.

8. USER INTERFACE & DESIGN SYSTEM

8.1 Design Philosophy

The refreshed interface uses a balanced indigo, teal, saffron, and rose palette with clear financial-status colors. Typography, spacing, borders, and compact surfaces prioritize scanning and repeated daily use. Decorative shadows are restrained, and data-entry dialogs avoid dark backdrop shading.

8.2 Design Tokens

The visual design is built on a set of design tokens — named values for colors, spacing, typography, shadows, and corner radii — defined centrally as CSS variables. Because every component references these tokens rather than hard-coded values, the entire appearance can be adjusted from one place, and the application can switch themes simply by changing the values the tokens point to.

8.3 Light and Dark Themes

The application supports light, dark, and system themes. The redesigned access screen uses a responsive two-panel layout, an India-focused INR identity, a deep indigo background, teal financial accents, a clear sign-in/register segmented control, and a focused form surface. On smaller screens, branding and forms collapse into a single unclipped layout while the theme control remains reachable.

8.4 Layout and Navigation

On larger screens, a fixed sidebar provides navigation between the main sections, each labeled with an icon. The top bar contains a theme toggle, a notifications bell, and an account menu. On smaller screens, the sidebar collapses into a hamburger menu that opens a drawer, ensuring that the application remains fully usable on mobile devices.

8.5 Reusable Components

- Application Shell — provides the consistent navigation and layout around every page.
- Stat Cards — display key summary figures on the dashboard and other pages.
- Progress Bars and Rings — visualize budget usage and savings progress.
- Transaction List — a shared, consistent list of transactions used across pages.
- Modals - responsive, page-scrollable add and edit windows with transparent overlays, wrapping controls, reachable actions, and no clipped fields.
- Charts — bar, line, pie, and area charts used in the dashboard and reports.

8.6 Responsiveness

The layout adapts fluidly across device sizes. Grids reflow from multiple columns on desktop to a single column on mobile, the navigation transforms into a drawer, and charts and tables resize to fit the available width. This ensures a comfortable experience whether the application is used on a phone, a tablet, or a computer.

9. APPLICATION WORKFLOW

This chapter describes the typical flow of a user through the application, from first use to regular daily use.

9.1 First-Time Use

25. The user opens the responsive RupeeFlow access screen and signs in, creates a local account, or starts verified password recovery.
26. For password recovery, the user enters the registered email, receives a six-digit OTP, verifies it, and sets a new password using the one-time reset authorization.
27. On successful registration, the user is signed in and taken to the dashboard, which is initially empty.
28. The user begins by adding income and expense transactions.

9.2 Daily Use

29. The user logs in and reviews the dashboard summary.
30. As money is earned or spent, the user records transactions in the income or expense screens.
31. The user checks budget status to ensure spending remains within limits.
32. The user reviews any notifications about budgets, bills, or goals.

9.3 Periodic Use

33. At the start of a month, the user sets or adjusts category budgets.
34. The user defines recurring rules for predictable income and expenses.
35. The user creates or updates savings goals and contributes to them.
36. The user opens the reports screen to analyze trends over the past months and exports reports if required.

9.4 Data Safety

37. Periodically, the user creates a JSON backup of their data from the settings screen.
38. If needed, the user restores data from a previous backup on the same or another device.

10. SECURITY CONSIDERATIONS

Because finance data is sensitive, the project separates current local-demo protections from production security claims. Finance records remain on the user's device, transaction integrity is protected by duplicate detection and validation, and backup/restore gives the user control over data portability.

10.1 Local Data Privacy

Financial collections are stored in IndexedDB on the current browser profile and are not transmitted to an application server. This improves local privacy for the academic demonstration, but device access and browser-profile security remain important. Users should create backups before clearing browser data.

10.2 Input Validation and Duplicate Integrity

Amounts, dates, categories, and required fields are validated before insertion. A normalized fingerprint derived from date, type, amount, category, and note prevents duplicate manual records, duplicate edits, repeated recurring records, and repeated statement imports.

10.3 Local Session Management

Account and session state remain local to the browser profile. Password recovery is separately protected by server-generated email OTP codes and one-time reset tokens. This is suitable for the academic demonstration, but public deployment should move complete identity and session management to a secure backend.

10.4 Account and Collection Separation

Duplicate email addresses and usernames are rejected within the local account registry. Finance collections use the user identifier in their storage keys, preventing records from being mixed between local accounts.

10.5 Statement-Import Safety

Statement import runs inside the browser. It accepts CSV and selectable-text PDF files, validates parsed rows, skips unsupported or invalid content, and reports when no transactions are found. Imported records still pass through duplicate checks before persistence.

10.6 Further Hardening for Public Deployment

Production deployment would require these additional controls:

- Move authentication and financial storage to a secure HTTPS backend with authorization on every request.

- Hash passwords with bcrypt or Argon2, protect secrets, and use secure session cookies or carefully managed tokens.
- Use a managed database with encryption, access controls, backups, and audit logging.
- Add broader rate limiting, multi-factor sign-in, security monitoring, audit logs, and automated dependency updates.

The existing separation between contexts, business logic, and the storage adapter supports this migration without redesigning every page.

11. TESTING & VALIDATION

Final verification covered code quality, production compilation, module calculations, statement parsing, duplicate identity, recurring generation, route availability, and OTP service health. No retained feature failed these automated checks.

11.1 Testing Approach

- Automated module checks covered totals, budget states, savings progress, trends, forecasts, duplicate fingerprints, recurring schedules, and INR formatting; all assertions passed.
- Statement parsing was validated with two real supported bank-statement samples: 127 transactions from the May statement and 235 transactions from the earlier statement were extracted successfully.
- OTP service health confirmed valid SMTP configuration; authenticated Gmail SMTP accepted a controlled delivery test, while invalid email, OTP, and reset-token requests were rejected correctly.
- Application routes for dashboard, transactions, budgets, goals, recurring rules, reports, and profile returned successfully through the development server. Custom-category persistence, viewport-safe modal rendering, and nonblank report printing were also checked after final UI fixes.
- ESLint completed without errors and Vite produced a successful optimized build from 611 transformed modules. The remaining large-chunk message is a performance advisory, not a functional failure.

11.2 Sample Test Cases

No	Test Case	Expected Result	Status
1	Register with valid local details	Account created and user can sign in on same browser profile	Pass
2	Register with existing email	Duplicate email rejected	Pass
3	Request password reset for registered email	Six-digit OTP accepted by SMTP service and sent to user	Pass
4	Enter incorrect, expired, or reused OTP	Reset denied with clear error; attempt and expiry rules enforced	Pass
5	Add valid expense	Expense saved in IndexedDB and totals update	Pass
6	Add identical transaction twice	Second entry rejected as duplicate	Pass
7	Edit a record to duplicate another	Edit rejected; original records preserved	Pass
8	Import supported selectable-text PDF	Supported samples extract 127 and 235 transactions; records are categorized and saved	Pass

No	Test Case	Expected Result	Status
9	Re-import same statement	Existing transactions are skipped by duplicate fingerprints	Pass
10	Import unsupported or scanned PDF	No-transactions-found guidance shown	Pass
11	Import CSV with duplicate rows	Valid unique rows added; duplicate rows skipped	Pass
12	Exceed a category budget	Budget status turns critical and alert appears	Pass
13	Add funds to savings goal	Progress and remaining amount update	Pass
14	Generate due recurring transaction	One due entry created without duplicate	Pass
15	Filter transactions by date/type/category	Only matching records shown	Pass
15a	Review dashboard forecast	Savings rate and 30-day projection calculate	Pass
16	Open add form on small viewport	All fields and action buttons remain scrollable and reachable	Pass
17	Backup and restore data	Collections restore accurately through storage API	Pass

11.3 Validation

Validation is applied throughout the application. Amounts must be positive, required categories and dates must be supplied, custom category names must be non-empty and unique, unsupported statement rows are skipped, and duplicate fingerprints are rejected. Import status messages report added and skipped counts, while reachable responsive dialog actions prevent incomplete entries caused by clipped forms.

12. SCREENSHOTS

The following screenshots show the completed RupeeFlow application using representative demonstration data.

Figure 12.1: Koushik R (U18ID23S0051) - Dashboard

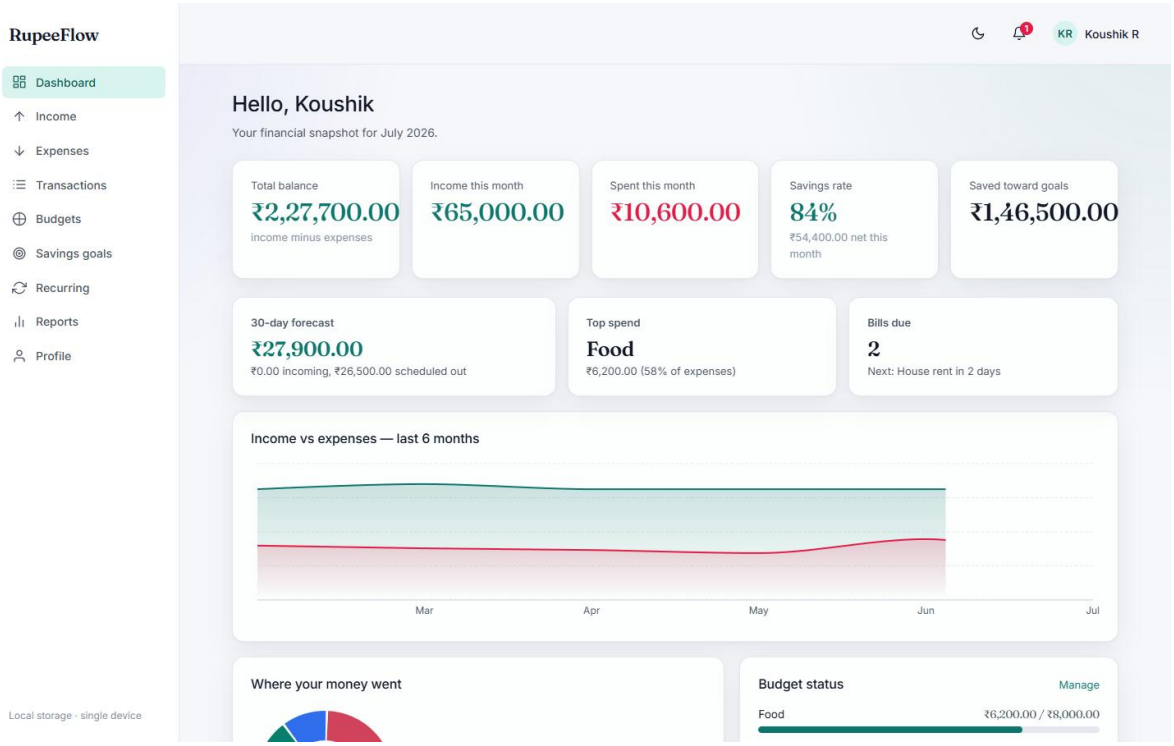


Figure 12.2: Koushik R (U18ID23S0051) - Transaction Management

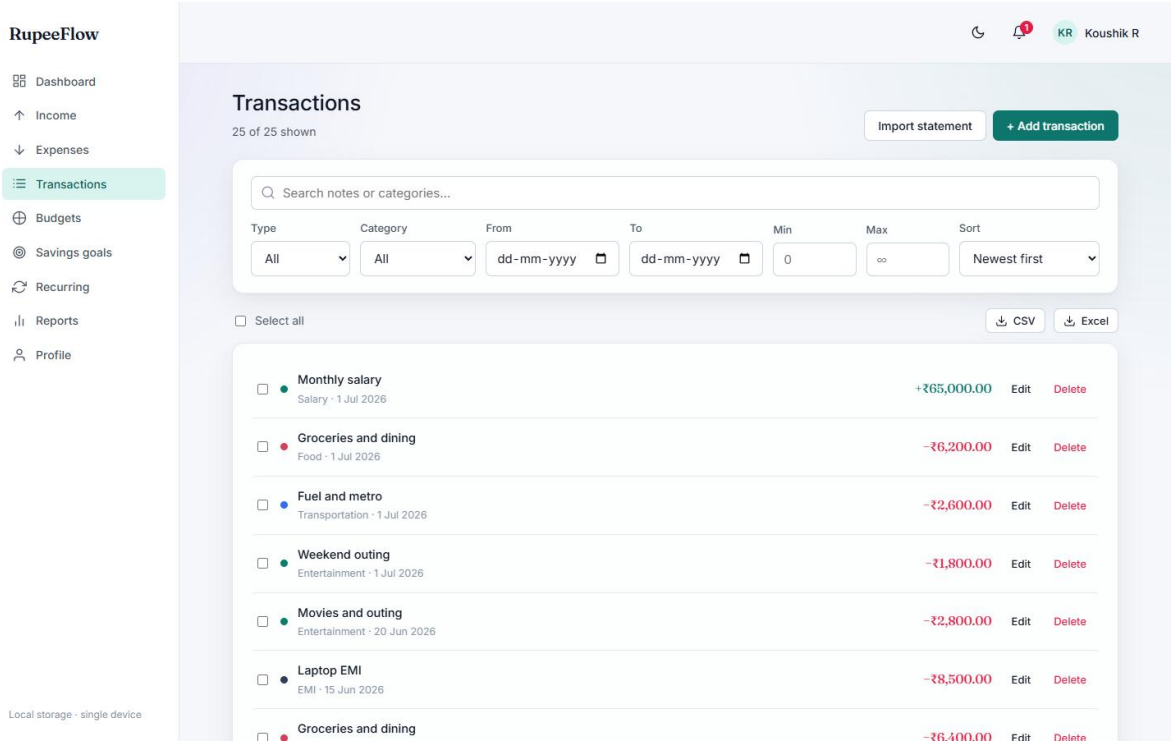


Figure 12.3: Koushik R (U18ID23S0051) - Budget Management

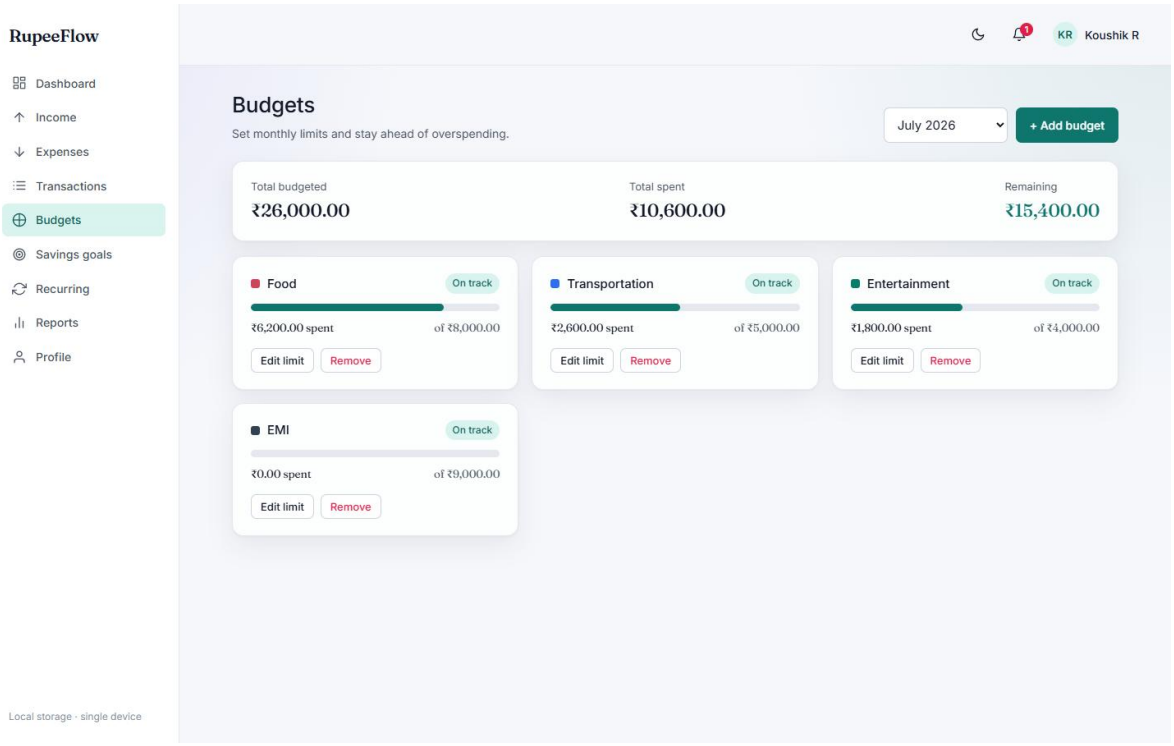


Figure 12.4: Koushik R (U18ID23S0051) - Savings Goals

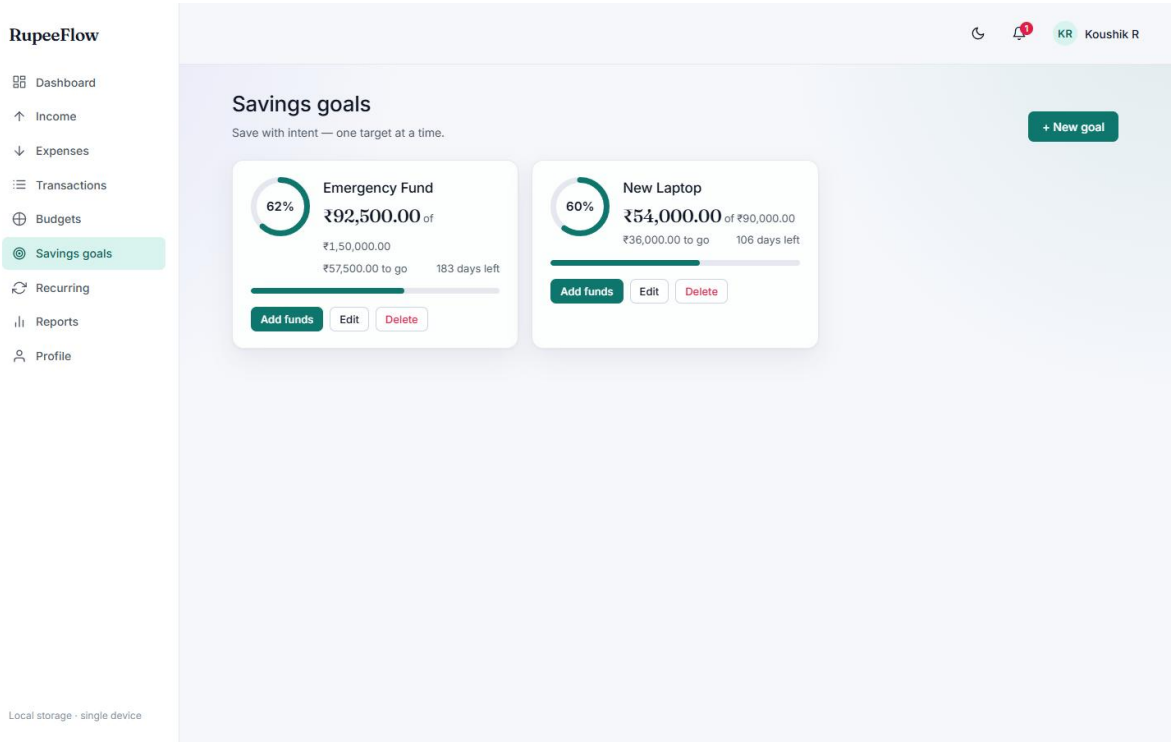


Figure 12.5: Koushik R (U18ID23S0051) - Reports and Analytics

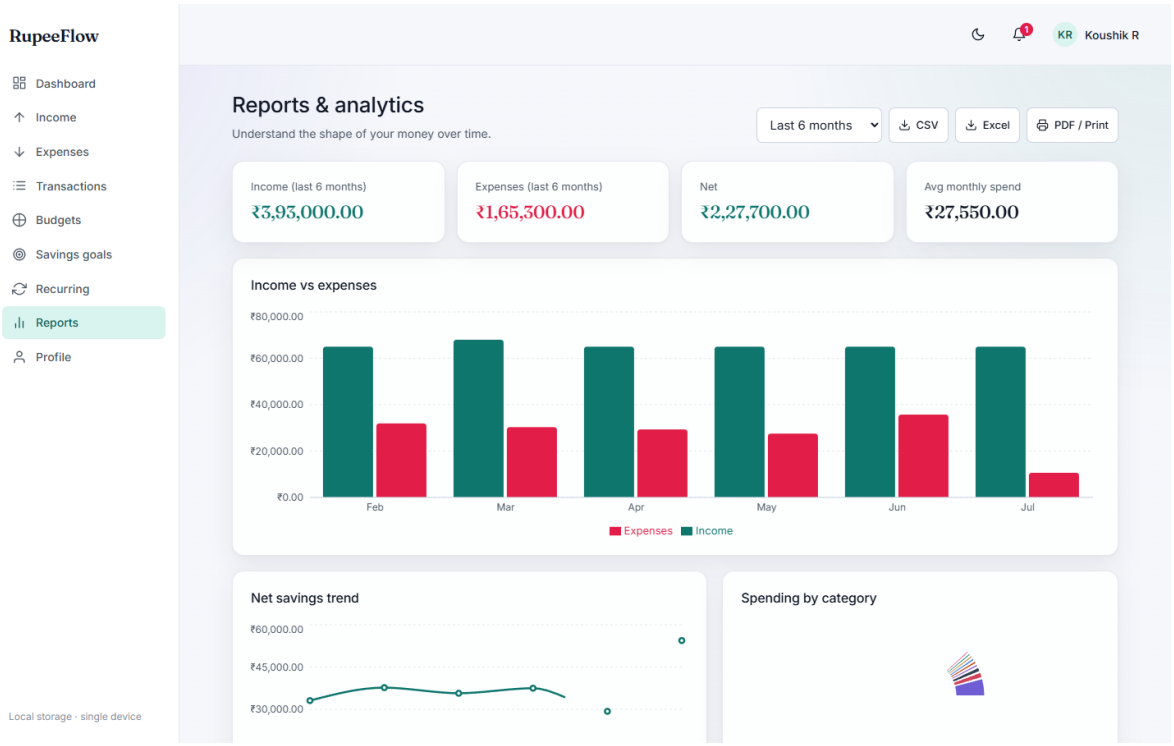
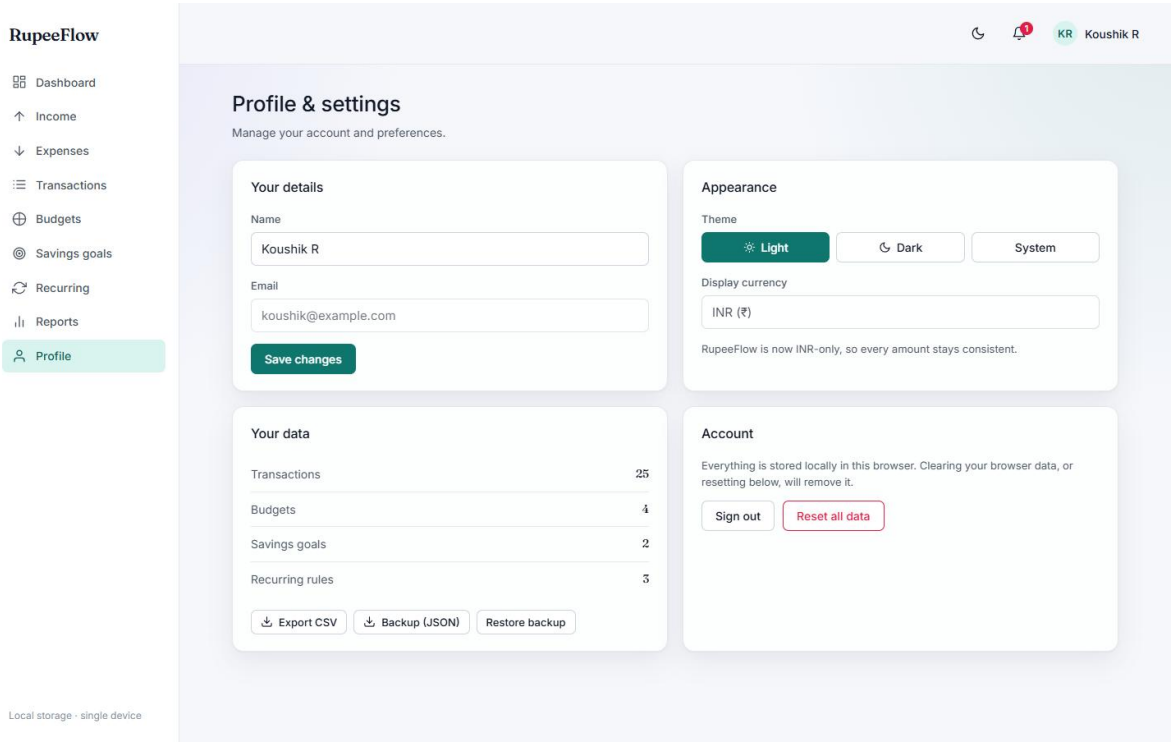


Figure 12.6: Koushik R (U18ID23S0051) - Profile and INR Settings



13. ADVANTAGES, APPLICATIONS & CONCLUSION

13.1 Advantages of the System

- Centralization — all financial information is kept in one organized place instead of being scattered across notebooks, spreadsheets, and bank apps.
- Automation — recurring transactions and automatic calculations remove repetitive manual effort.
- Awareness — clear dashboards, category breakdowns, and trends help the user understand their spending habits.
- Discipline — budgets and alerts actively discourage overspending, while savings goals encourage consistent saving.
- Data integrity - validation and duplicate fingerprints prevent repeated transactions from manual entry, recurring generation, editing, or imports.
- Accessibility — the responsive design works comfortably on phones, tablets, and computers, with light and dark themes.
- India practicality - INR-only display, UPI/EMI-aware categories, and supported PDF bank-statement import reduce manual work.
- Extensibility - contexts, pure logic modules, and a storage adapter allow future secure cloud persistence without replacing feature pages.

13.2 Applications of the System

- Personal budgeting for students managing limited monthly allowances.
- Expense and savings tracking for working professionals.
- Income management for freelancers with variable earnings from multiple clients.
- Basic personal financial record-keeping for small business owners.
- A learning tool for anyone wishing to build better financial habits.

13.3 Conclusion

RupeeFlow Finance Tracker achieves its objective of providing a practical INR-first platform for personal finance. It combines transaction management, PDF and CSV statement import, duplicate prevention, budgets, goals, recurring entries, notifications, forecasts, and visual reporting in one responsive application.

Technically, the project demonstrates component-based React design, centralized state, pure calculation and parsing modules, asynchronous IndexedDB persistence, backward-compatible migration, PDF.js integration, and responsive theme-aware UI engineering. Clear boundaries between presentation, state, logic, and storage make the application maintainable and extensible.

In conclusion, the RupeeFlow Finance Tracker is both a practical tool for everyday financial management and a demonstration of well-structured web-application design. It lays a solid

foundation upon which more advanced capabilities can be built in the future. Through the course of building this project, valuable experience was gained in user-interface design, application-state management, data modeling, and the disciplined structuring of a non-trivial software system.

14. LIMITATIONS & FUTURE ENHANCEMENTS

14.1 Limitations

- Financial data is local to one browser profile, so it does not automatically synchronize across devices.
- Local browser authentication does not synchronize across devices; password-reset email also requires valid private SMTP configuration on the OTP service.
- PDF import requires an unlocked statement with selectable text and currently supports known statement layouts; scanned image-only PDFs require OCR, which is not included.
- The application currently runs on a local development server; a public deployment would require hosting and HTTPS setup.

14.2 Future Enhancements

The architecture has been designed to make the following enhancements straightforward to add:

- Add a secure backend with managed PostgreSQL, MongoDB, Firebase, or Supabase storage and multi-device synchronization.
- Move complete account authentication to a secure backend and optionally add Google sign-in while retaining verified email recovery.
- Add opt-in reminders for EMIs, utility bills, budget limits, and savings goals.
- Add explainable spending insights, anomaly detection, and personalized savings suggestions.
- Extend forecasting with recurring obligations, seasonal patterns, and configurable planning scenarios.
- Add OCR for scanned bank statements and bank-specific parsers for major Indian banks such as SBI, HDFC, ICICI, Axis, Canara, Kotak, PNB, Bank of Baroda, and Union Bank. Add format detection, import confidence review, receipt scanning, and optional consent-based bank integrations. Universal accuracy cannot be guaranteed because statement layouts vary and change.
- Advanced modules such as investment tracking, loan and EMI tracking, and shared or family accounts.

These enhancements would extend the application from a capable personal tool into a comprehensive financial-management platform, while building directly on the existing, well-structured foundation.

15. BIBLIOGRAPHY

39. React Documentation — https://react.dev
40. MDN Web Docs (HTML, CSS, JavaScript) — https://developer.mozilla.org
41. Vite Documentation - https://vite.dev
42. Express Framework Documentation — https://expressjs.com
43. Vite Documentation — https://vitejs.dev
44. Recharts Documentation — https://recharts.org
45. JavaScript: The Definitive Guide, David Flanagan, O'Reilly Media.
46. Eloquent JavaScript, Marijn Haverbeke — https://eloquentjavascript.net
47. IndexedDB API Reference - https://developer.mozilla.org/en-US/docs/Web/API/IndexedDB_API
48. PDF.js Documentation - https://mozilla.github.io/pdf.js/
49. pdfjs-dist Package - https://www.npmjs.com/package/pdfjs-dist